

CS-218 DATA STRUCTURES AND ALGORITHMS

LECTURE 24

BY UROOJ AINUDDIN



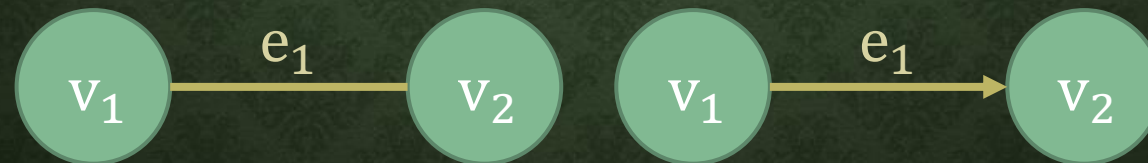
GRAPHS

BOOK 4 CHAPTER 8



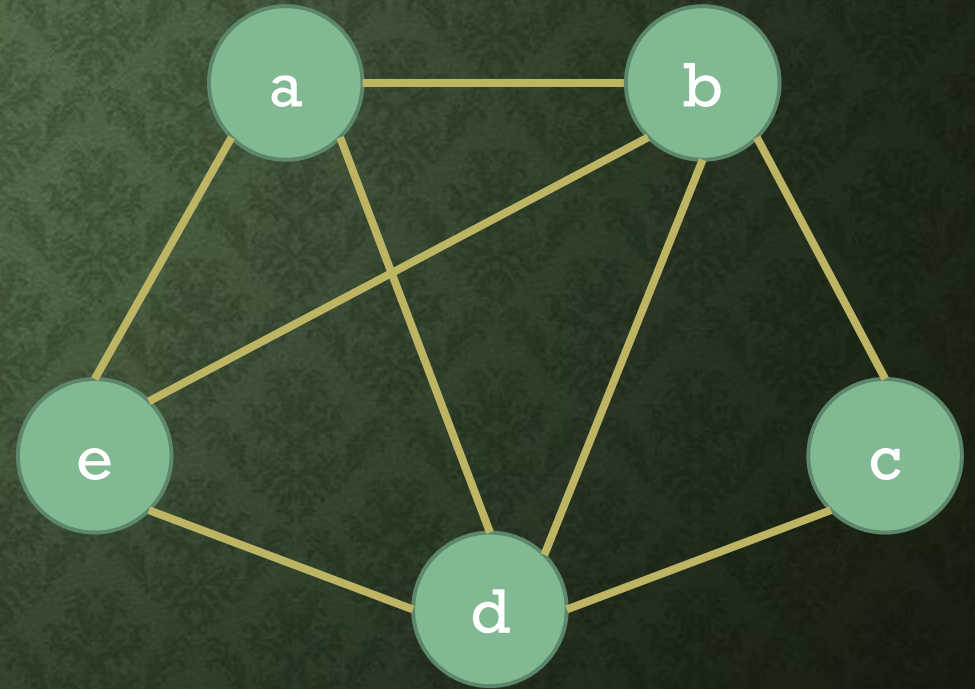
DEFINITION

- A **graph** G is a mathematical structure which comprises two sets: a set V , the set of vertices or nodes, and a set E , the set of edges.
- An **edge** e_1 from the set E connects two nodes v_1 and v_2 , both of which come from the set V .
- If the edges are undirected, we refer to G as an **undirected graph** or simply a graph.
- If the edges are directed, we refer to G as a **directed graph** or a digraph.
- If E contains both directed and undirected edges, we call G a **mixed graph**.



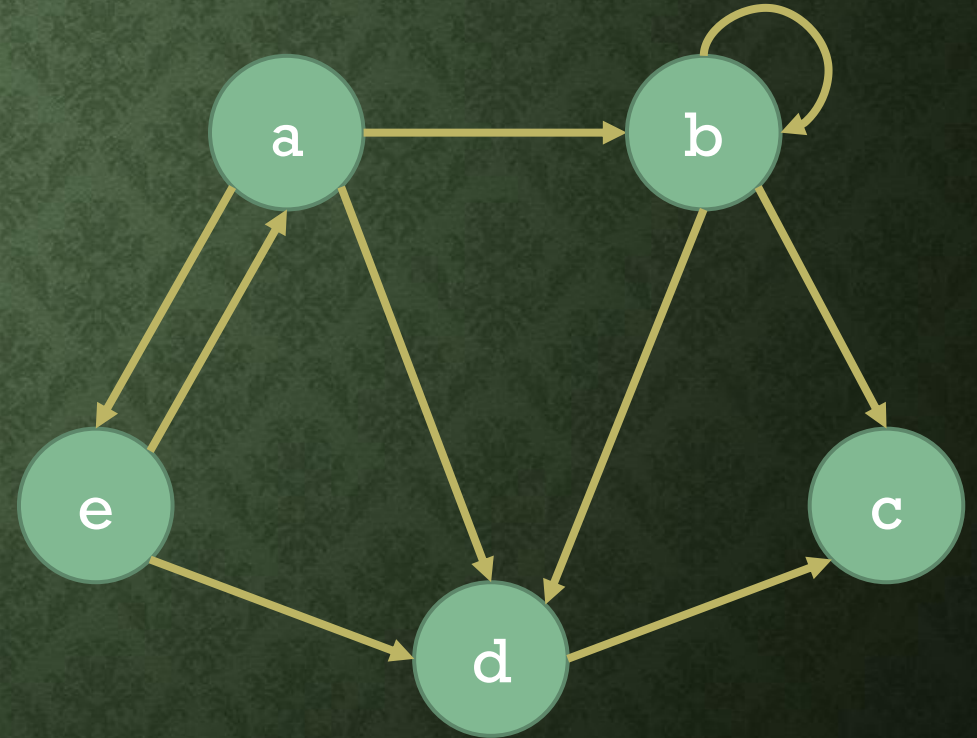
TERMINOLOGY

- Two vertices or nodes are **adjacent** if they are connected by an edge.
- In an undirected graph, each vertex v has a **degree**, $\deg(v)$, which is the number of edges connected to v .
- What is the degree of the vertices in the graph shown here?



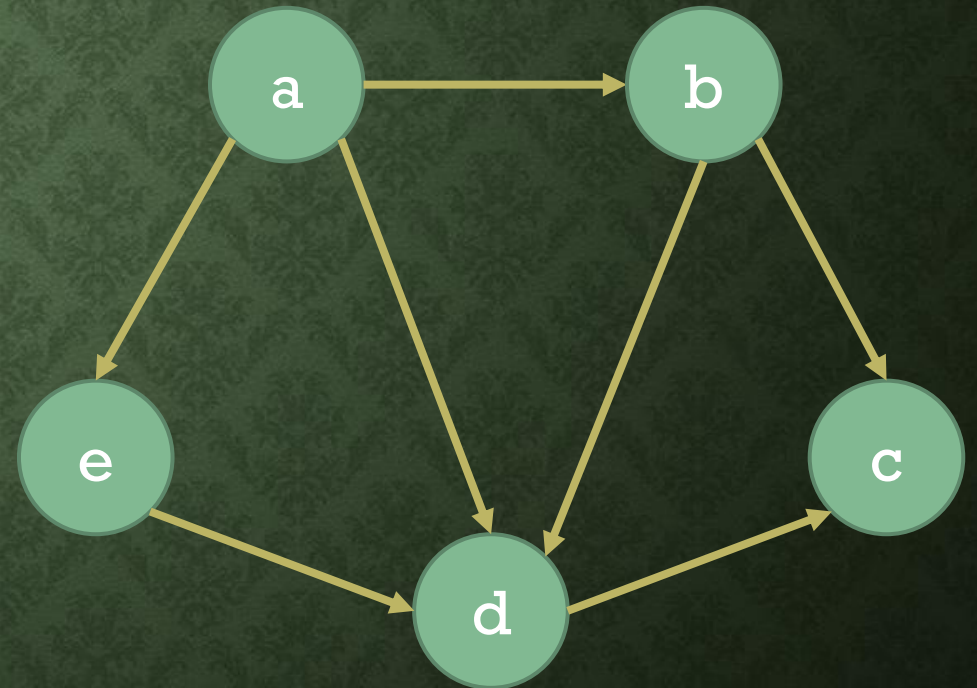
TERMINOLOGY

- In a directed graph, each vertex v has an **indegree**, $\text{indeg}(v)$, which is the number of edges coming into v .
- In a directed graph, each vertex v has an **outdegree**, $\text{outdeg}(v)$, which is the number of edges going out of v .
- What are the indegree and outdegree of the vertices in the digraph shown here?



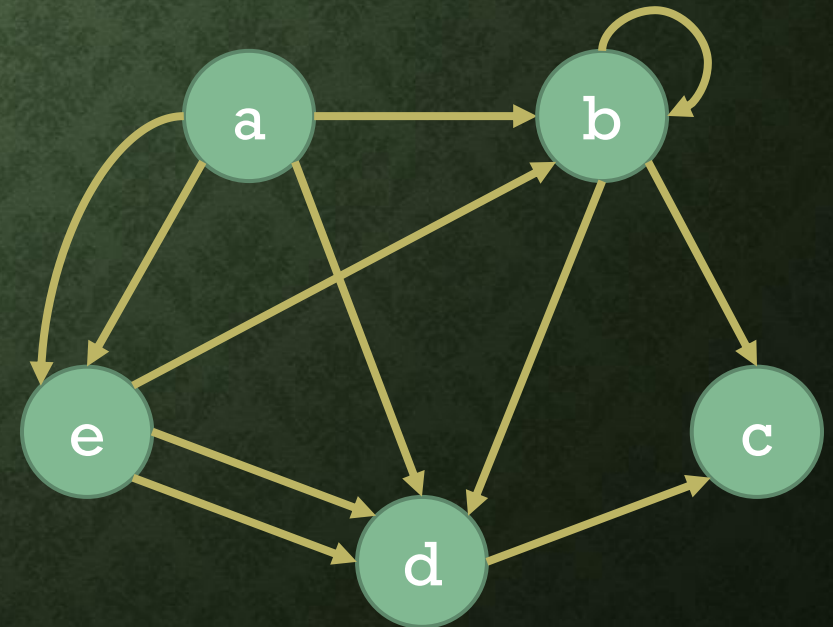
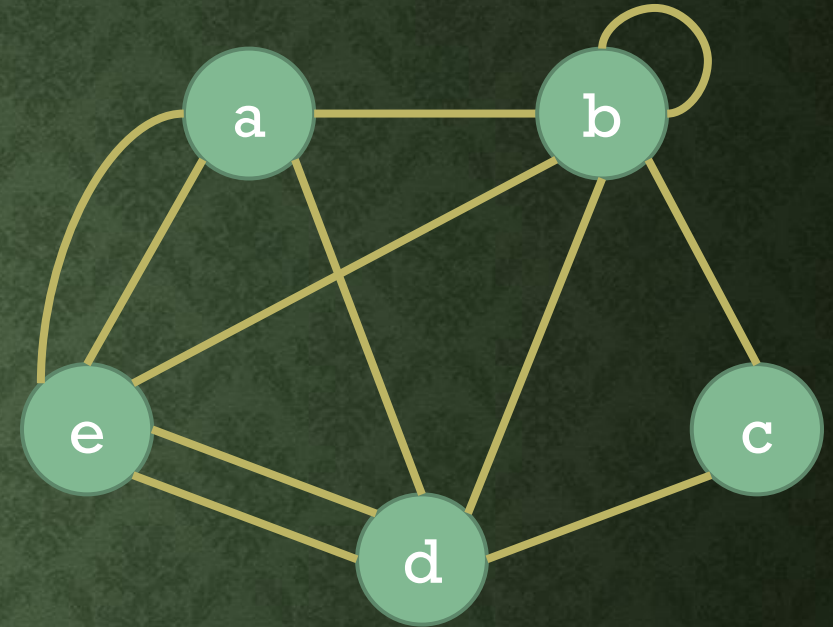
TERMINOLOGY

- A vertex with no incoming edge is a **source**.
- A vertex with no outgoing edge is a **sink**.



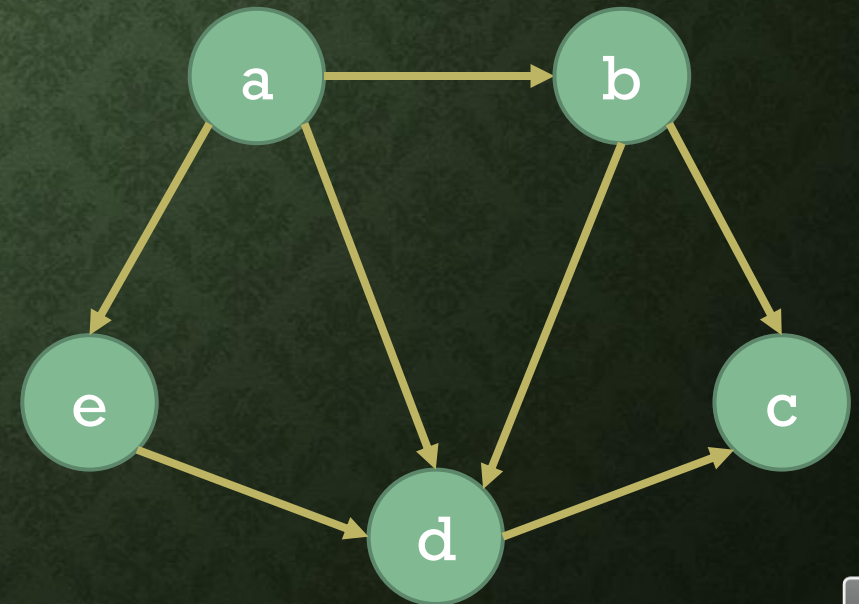
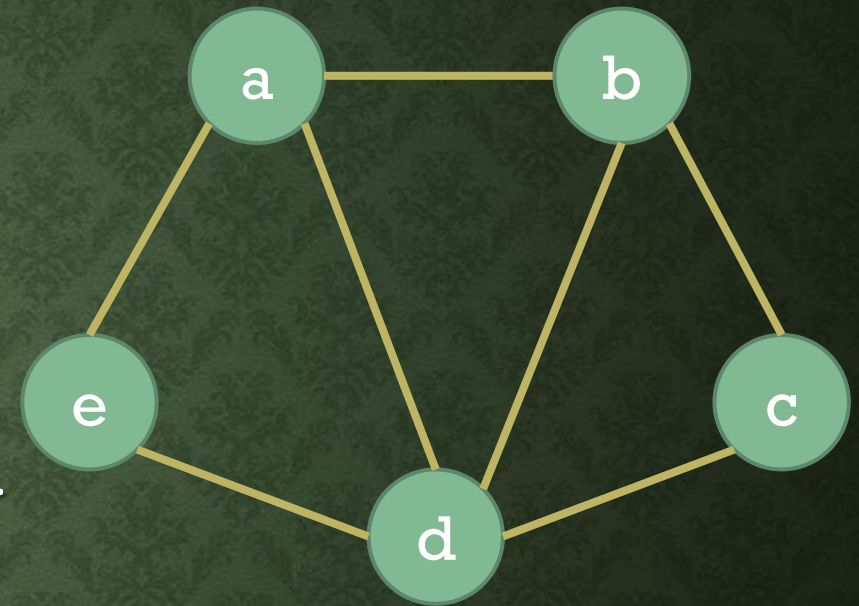
PARALLEL EDGES AND SELF-LOOPS

- In an undirected graph, if two edges connect the same vertices u and v , or in a digraph, if two edges go from the vertex u to the vertex v , these edges are called **parallel** or **multiple** edges.
- An edge that connects the vertex v to itself is called a **self-loop**.
- A **simple** graph does not have parallel edges or self-loops.



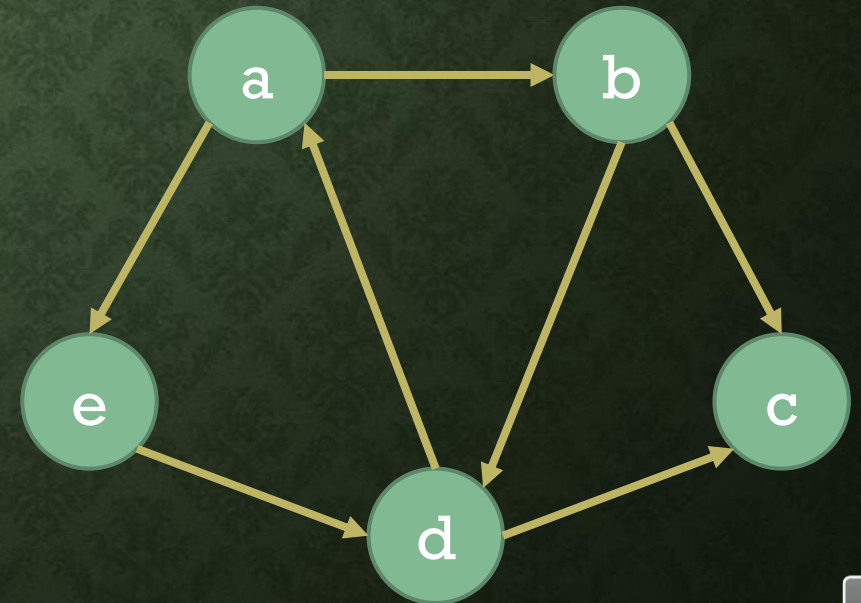
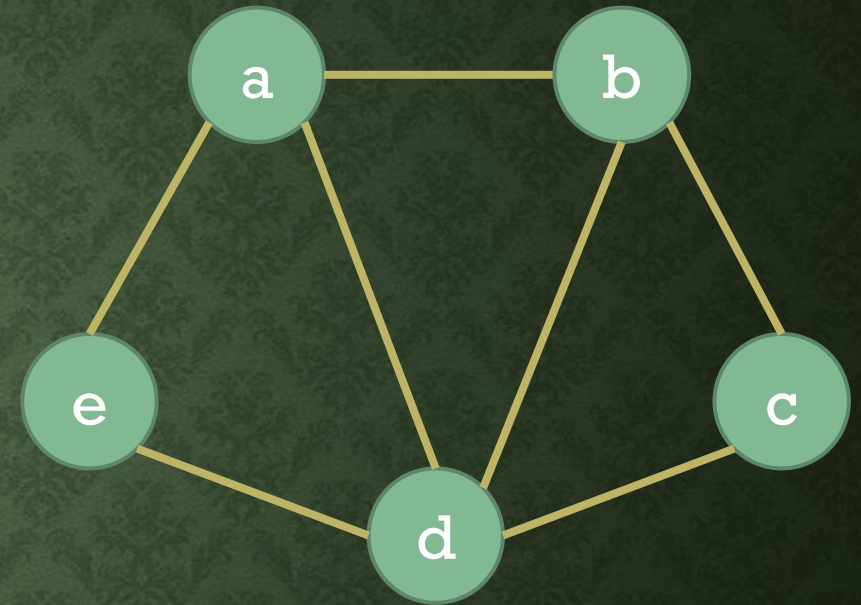
PATH

- A **path** from vertex v_1 to vertex v_2 is a sequence of vertices: $\langle v_1, v_i, v_{i+1}, \dots, v_m, v_2 \rangle$ such that v_1 is adjacent to v_i , v_i is adjacent to $v_{i+1}, \dots$ and v_m is adjacent to v_2 .
- The **length** of a path is the number of edges that constitute it.
- The length of a path is one less the number of nodes in the path.



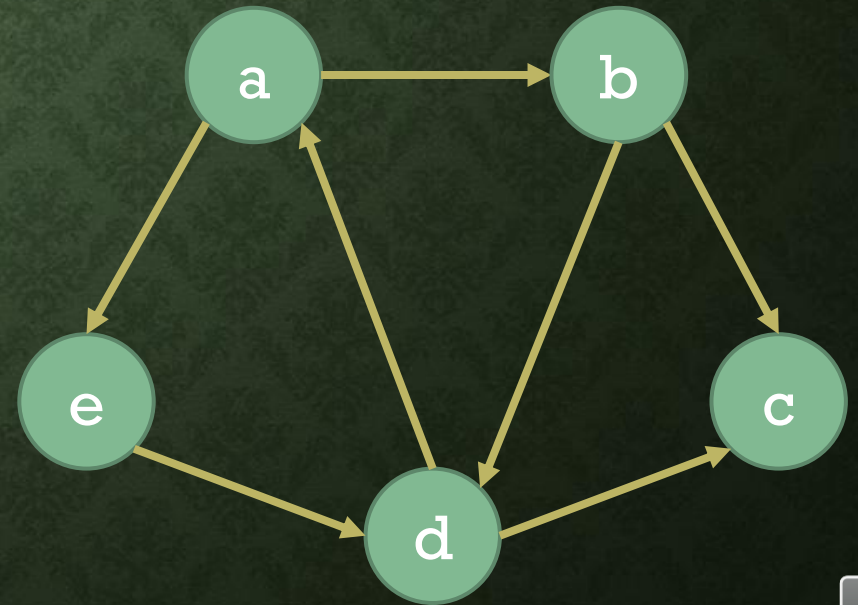
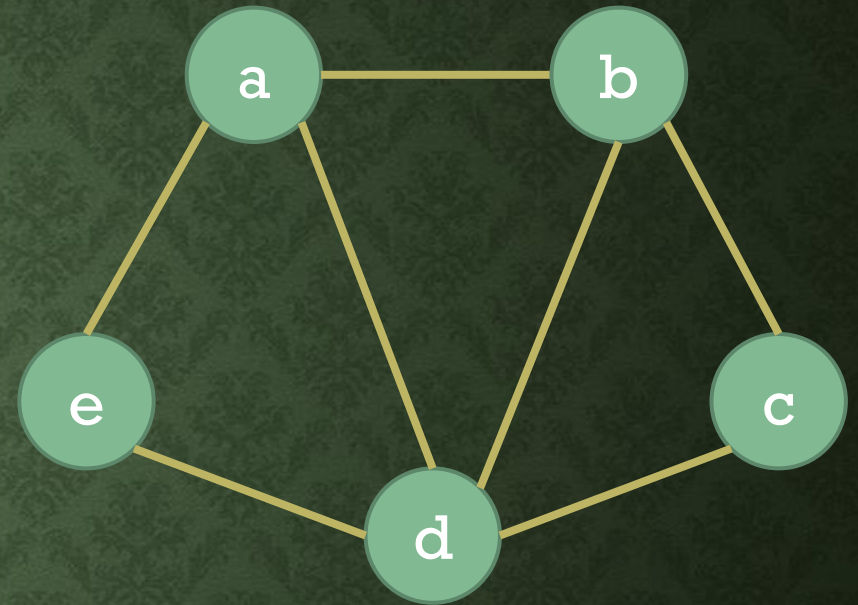
PATH

- A path composed of undirected edges is an **undirected** path.
- A path composed of directed edges is a **directed** path.
- A path is **simple** if no vertex is visited twice when moving along the path.
- We concern ourselves with simple paths only.



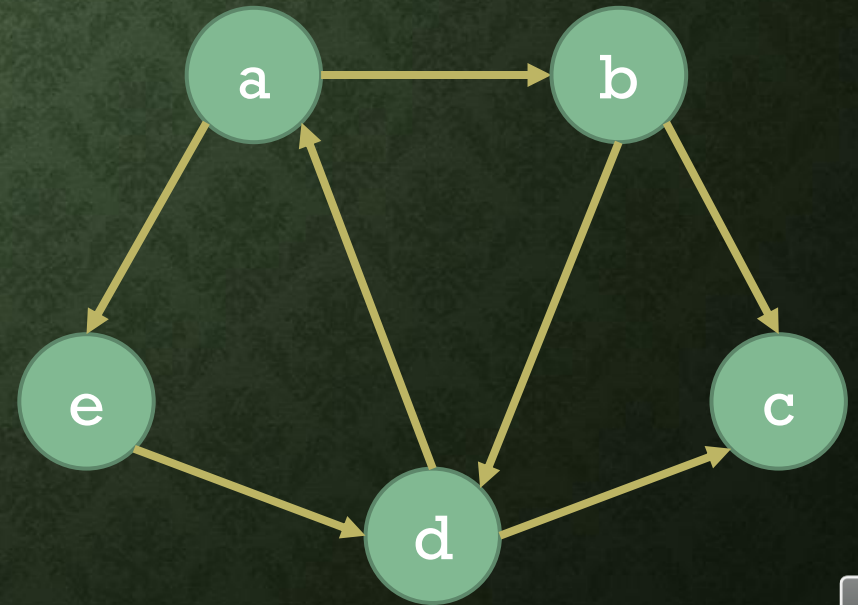
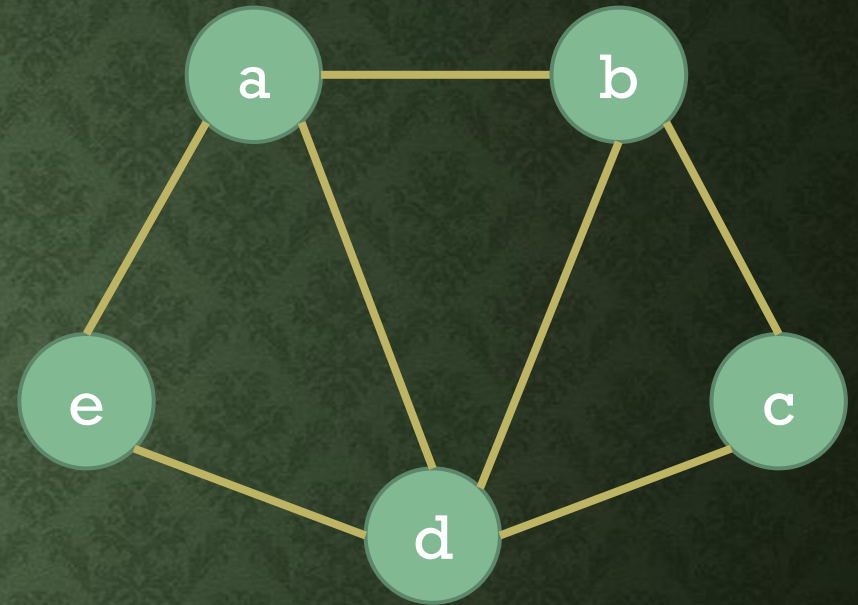
CYCLE

- A **cycle** is a simple path of length greater than 2 that starts and ends at the same vertex.
- No node is repeated in a cycle, except the first node, which is also the last node.
- A graph or a digraph is **acyclic** if no cycle is present.



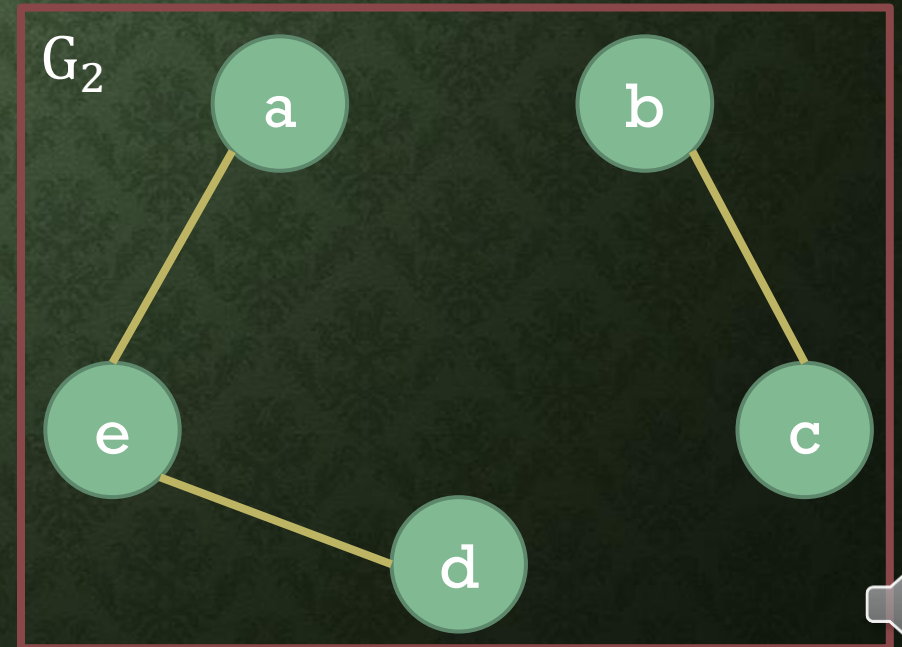
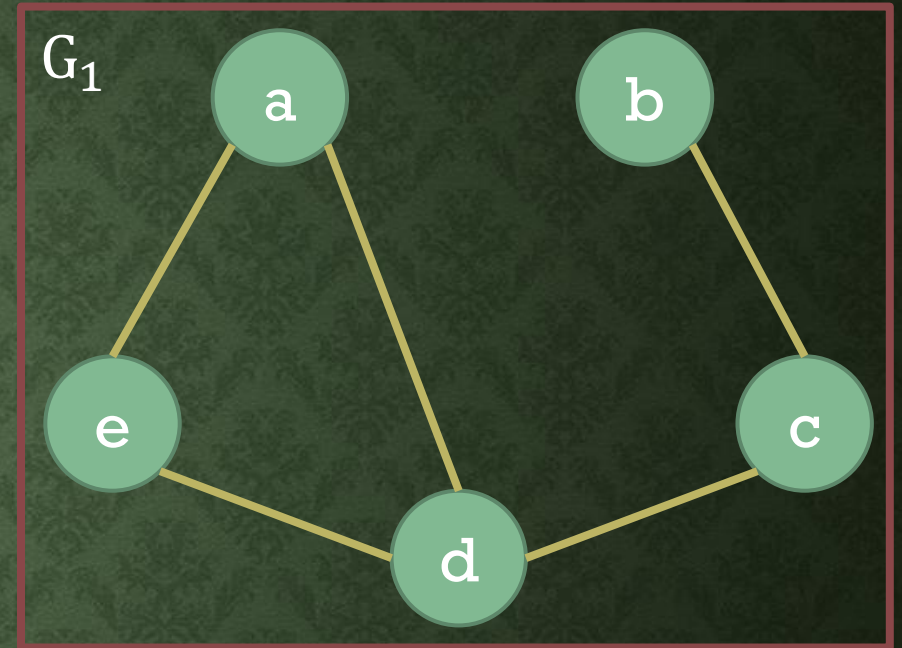
REACHABILITY

- A vertex **v** is **reachable from u** if there is a path from u to v. We say **u reaches v**.
- In an undirected graph, reachability is symmetric. If u reaches v, then v reaches u.
- In a digraph, it is possible that u reaches v, but v does not reach u.



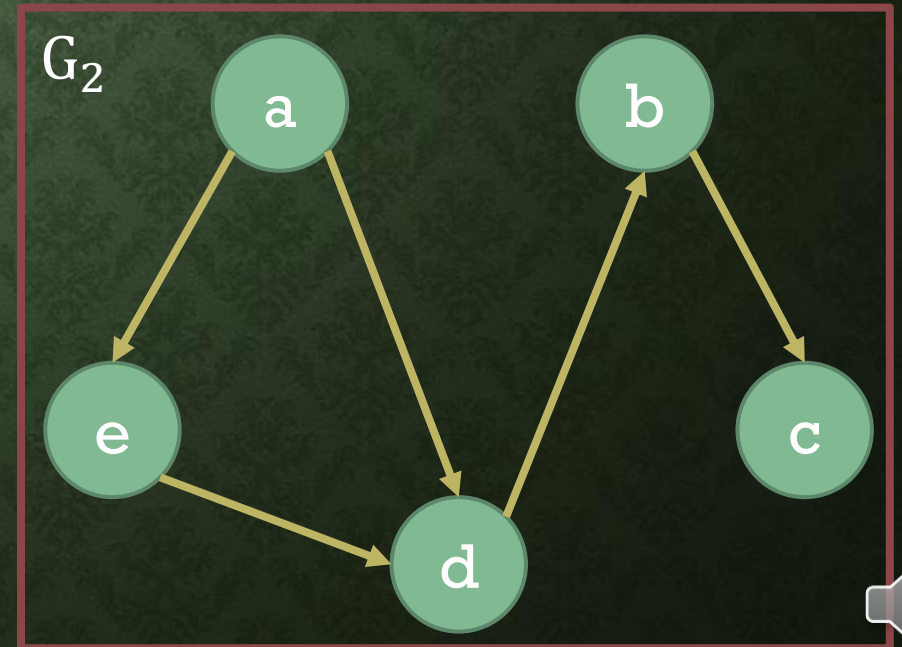
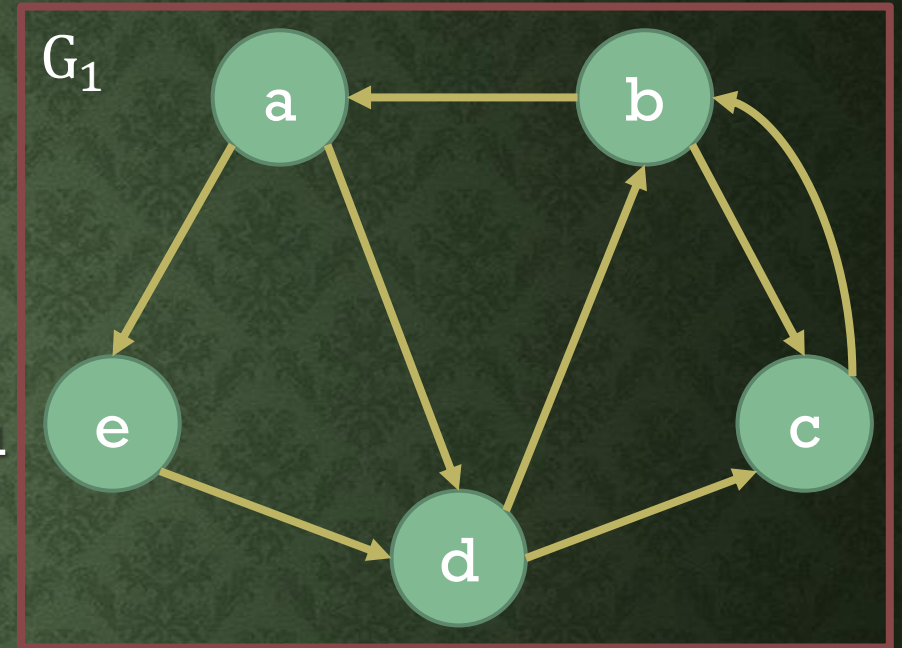
CONNECTED GRAPH

- A graph G is **connected** if all vertices in G are reachable from all vertices.
- The complete traversal of a disconnected graph is impossible.
- The extent of traversal of a disconnected graph depends on the node from which traversal starts.



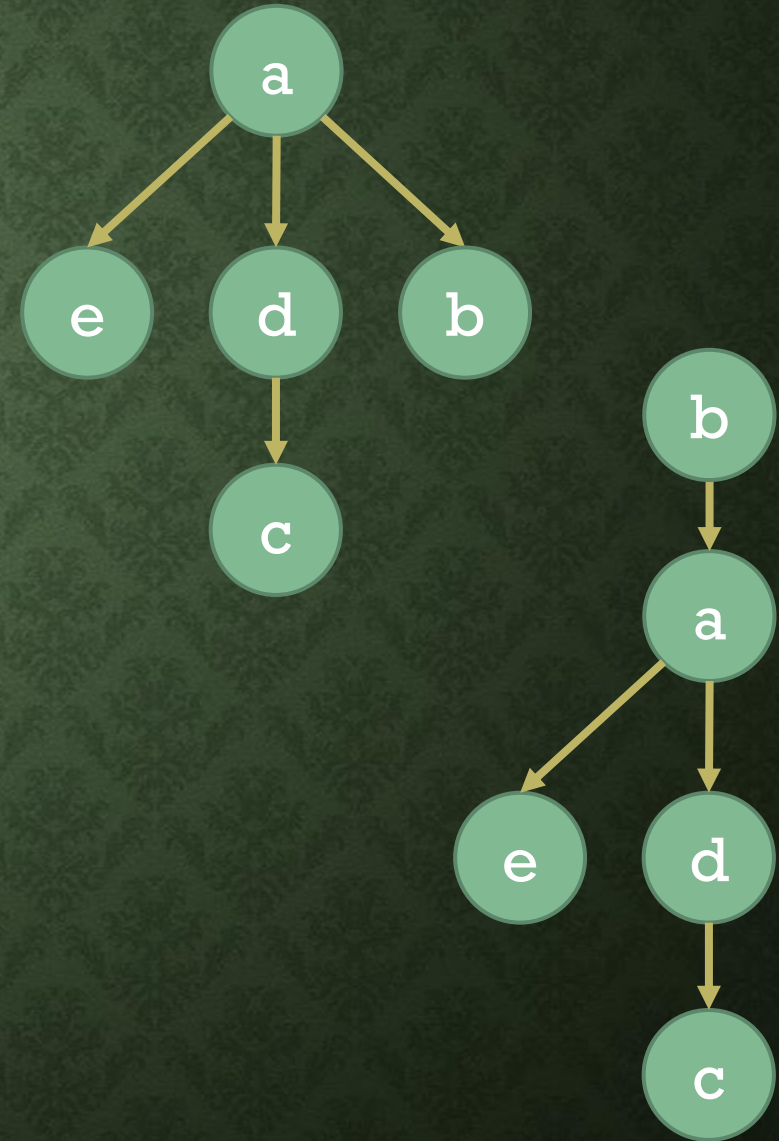
CONNECTED DIGRAPH

- A digraph G is **strongly connected** when, if u reaches v , then v reaches u .
- If a digraph G is not strongly connected, but by replacing all directed edges with undirected edges, we get a connected graph, then G is **weakly connected**.



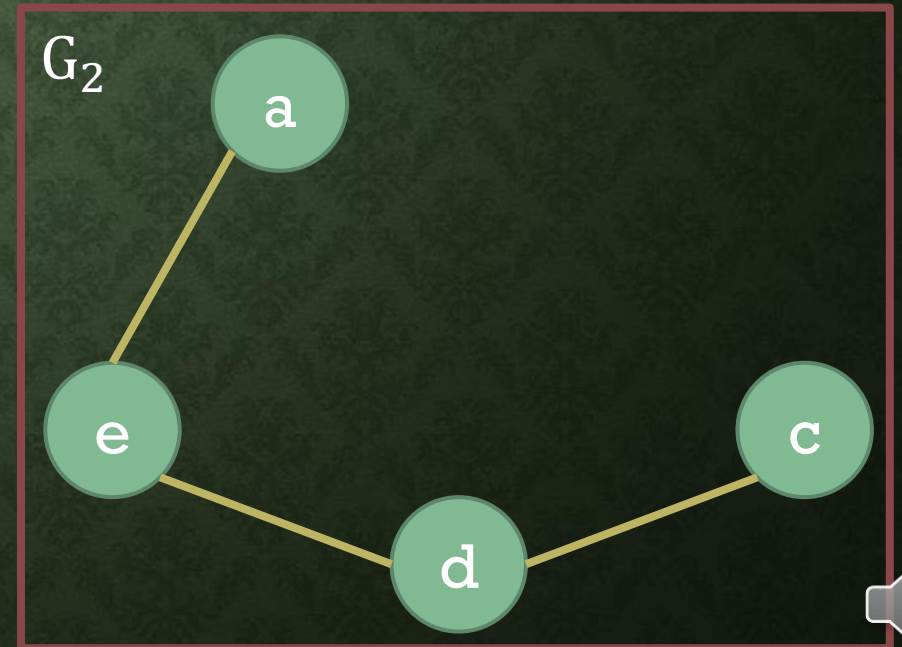
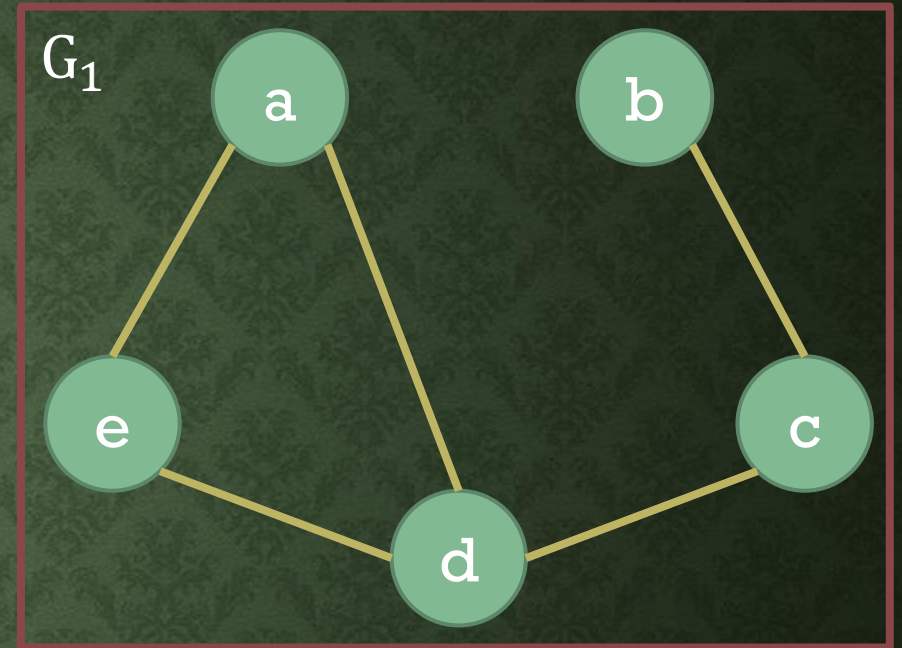
TREE

- A tree is an **acyclic connected digraph** with n nodes and $n - 1$ edges.
- Proof: A tree has n nodes. Every node has an edge coming into it from its parent node, except the root. So, there are $n - 1$ edges.



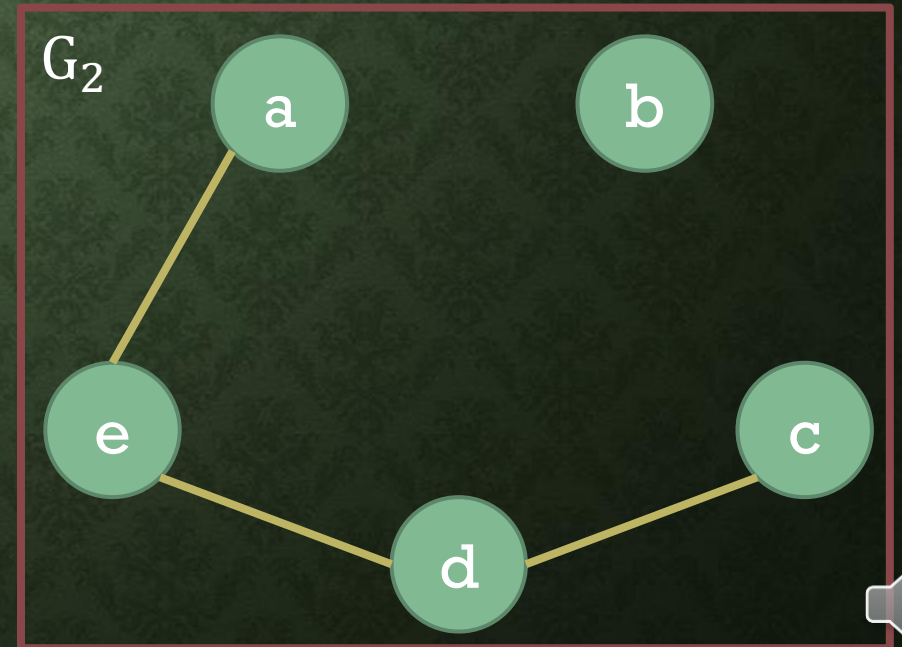
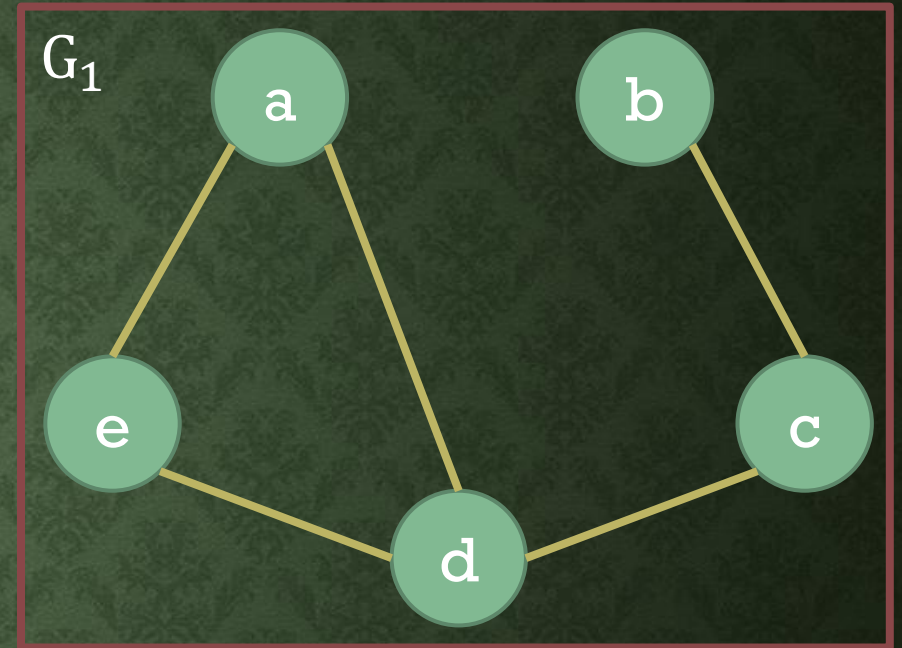
SUBGRAPH

- A **subgraph** H of graph G has a set of vertices that is the subset of that in G , and a set of edges that is a subset of that in G .



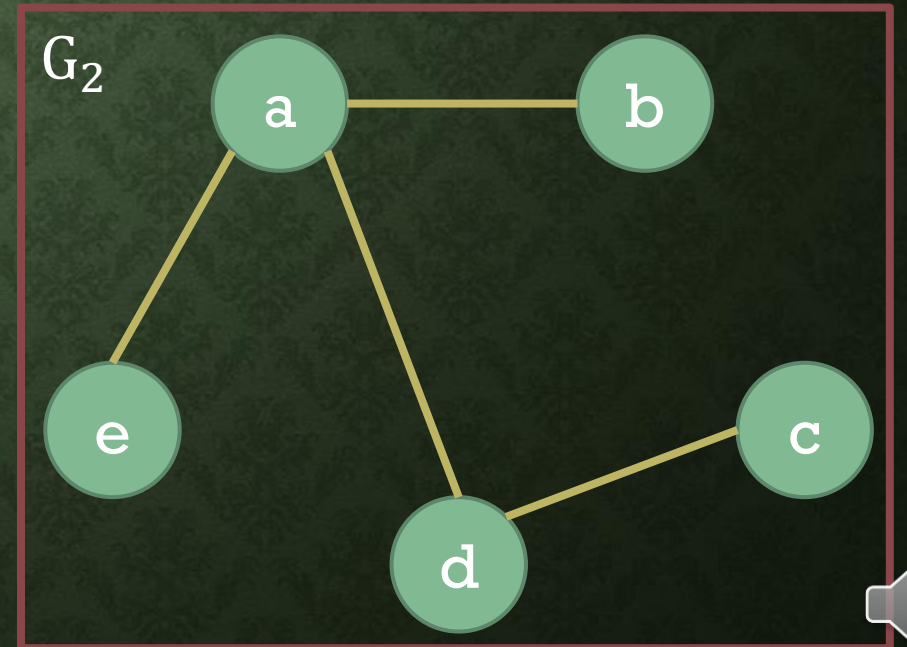
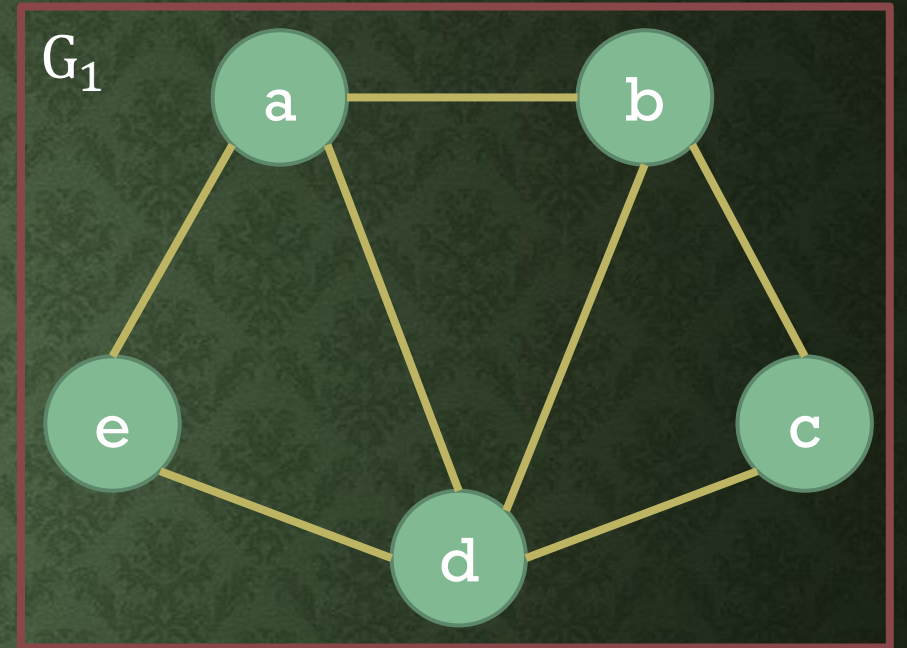
SPANNING SUBGRAPH

- A **spanning subgraph** H of graph G has the same set of vertices as G , and a set of edges that is a subset of that in G .



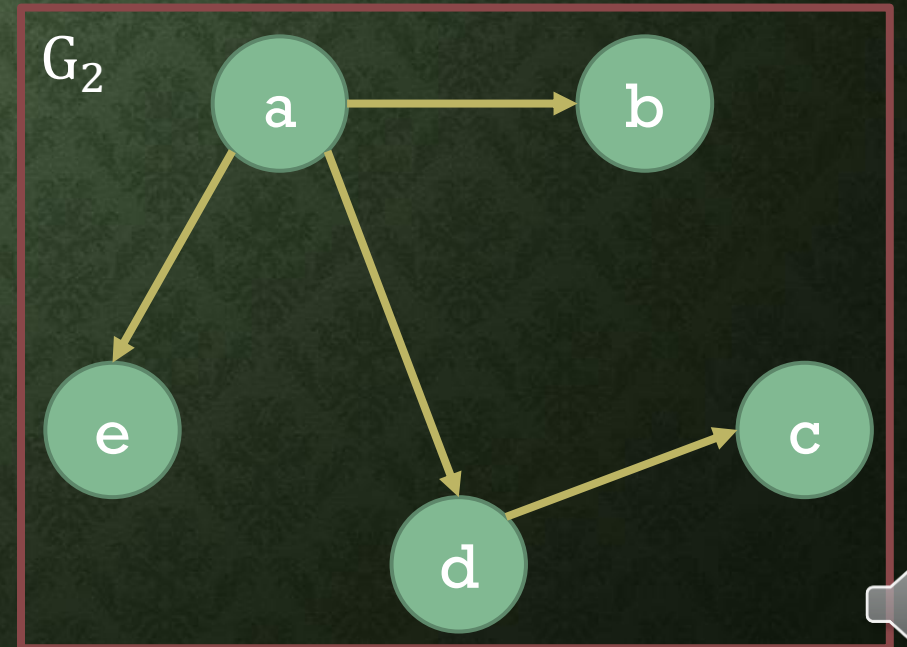
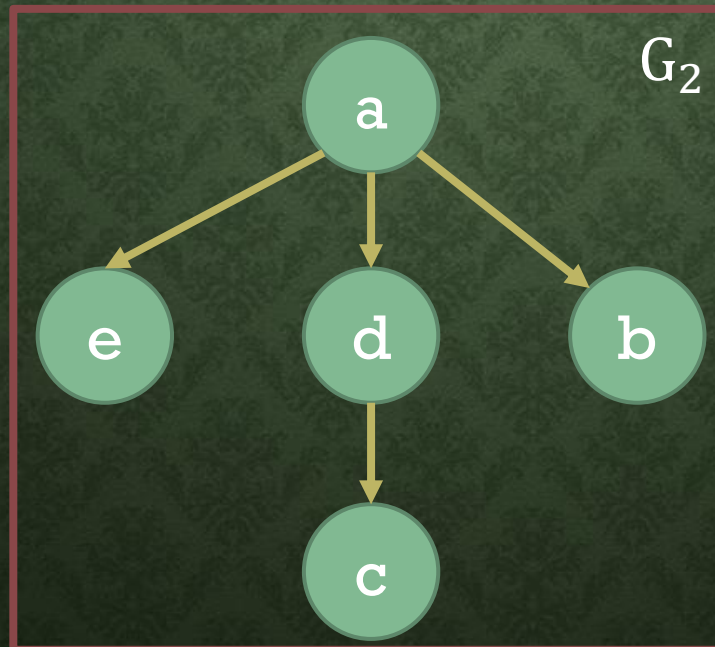
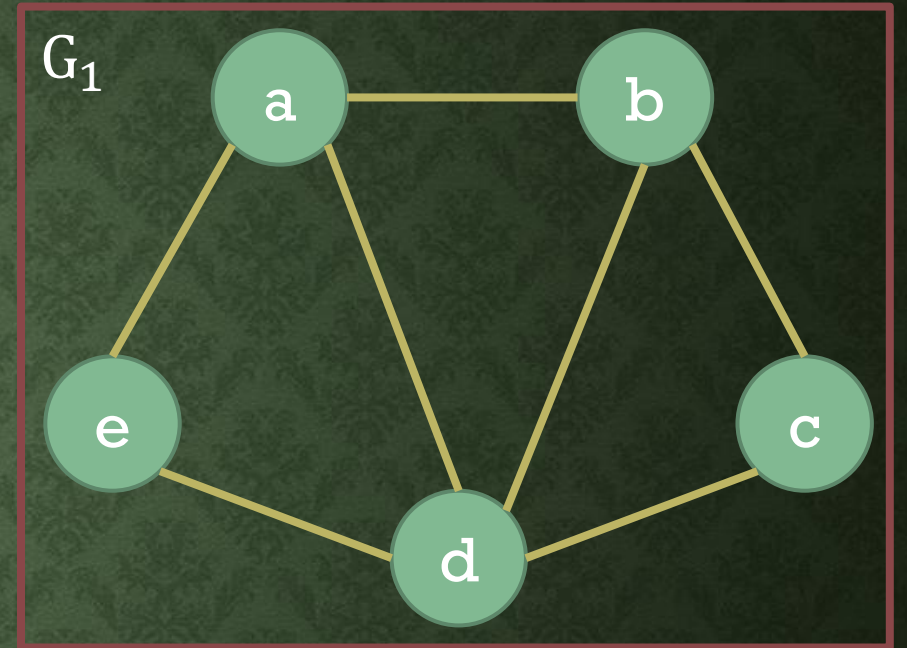
SPANNING TREE

- A **spanning tree** of graph G is a spanning subgraph that is a tree.



SPANNING TREE

- A **spanning tree** of graph G is a spanning subgraph that is a tree.



DATA STRUCTURES FOR GRAPHS

Storing graphs in memory



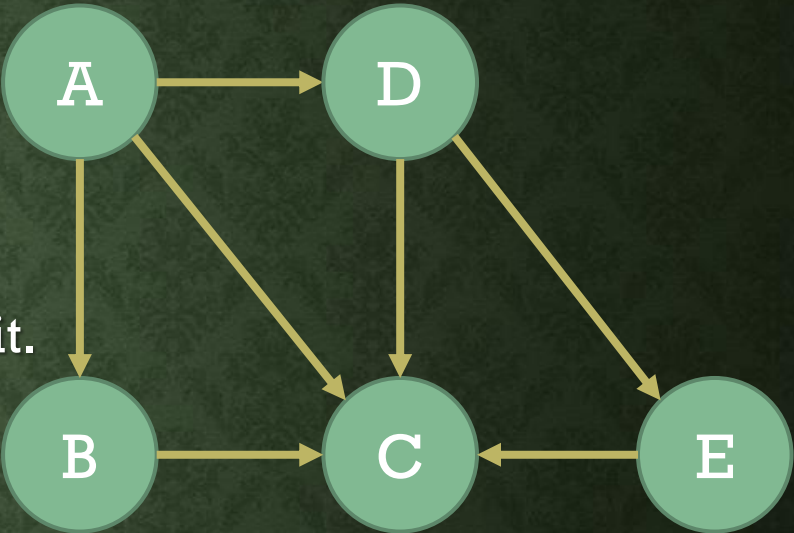
GRAPH STORAGE TECHNIQUES

- There are two approaches for storage of graphs in memory:
 1. Adjacency list,
 2. Adjacency matrix.

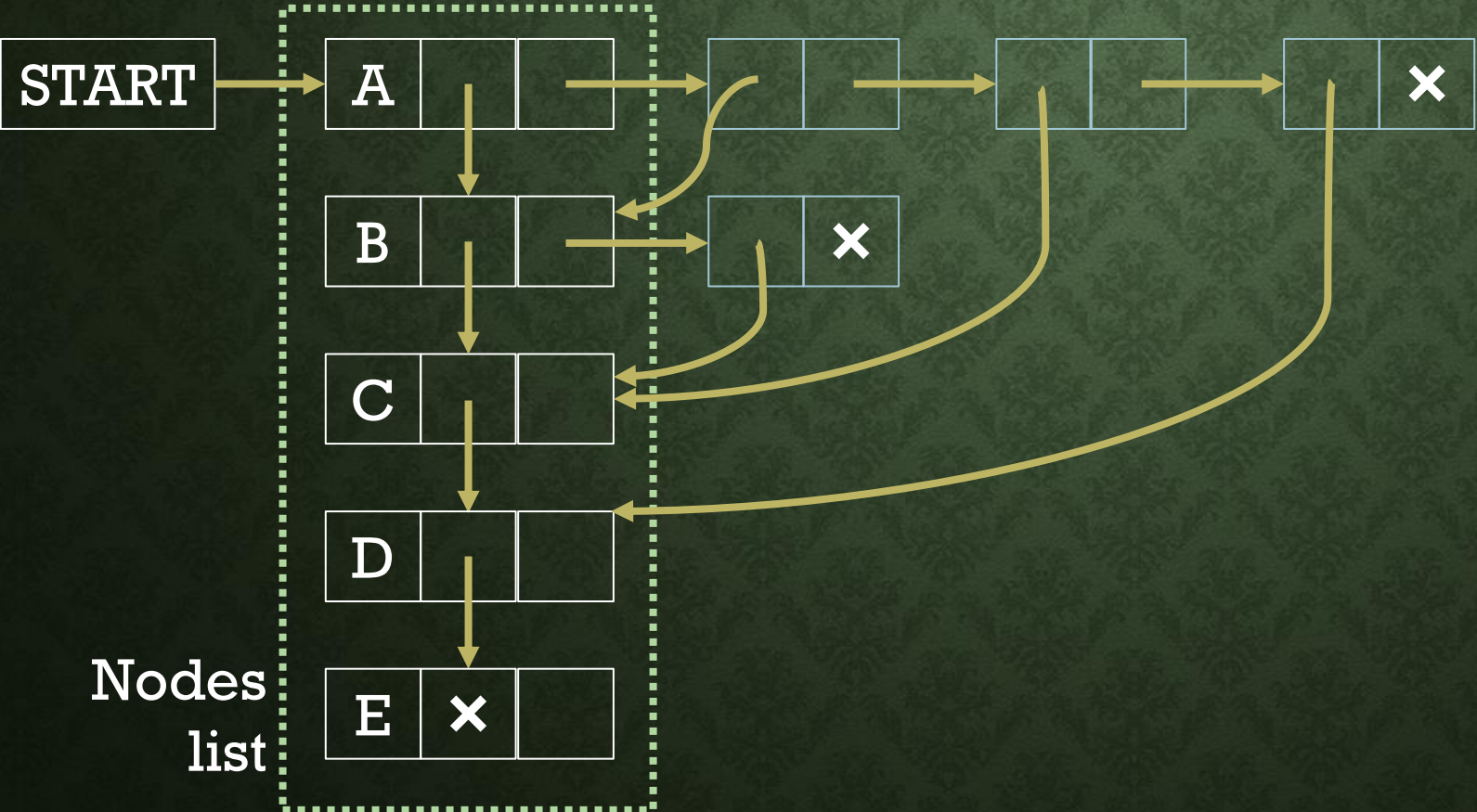


node	next	adj
------	------	-----

ADJACENCY LIST



- For each vertex, maintain a list of nodes that are adjacent to it.

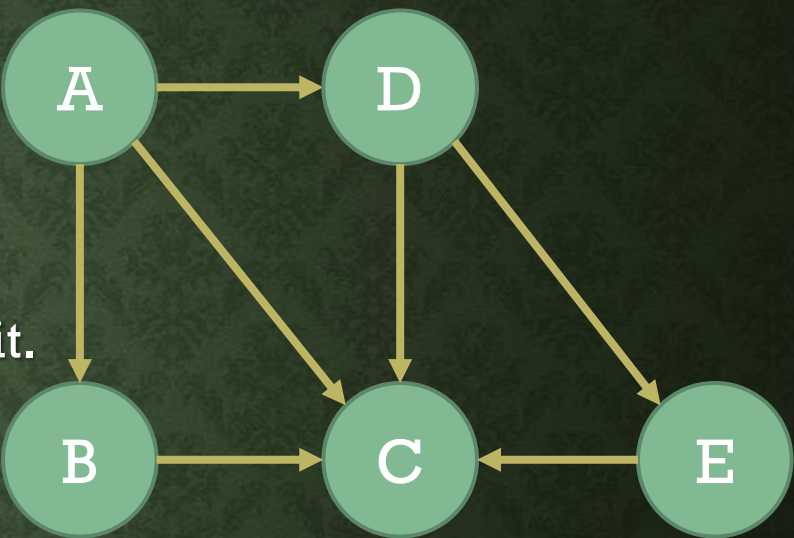
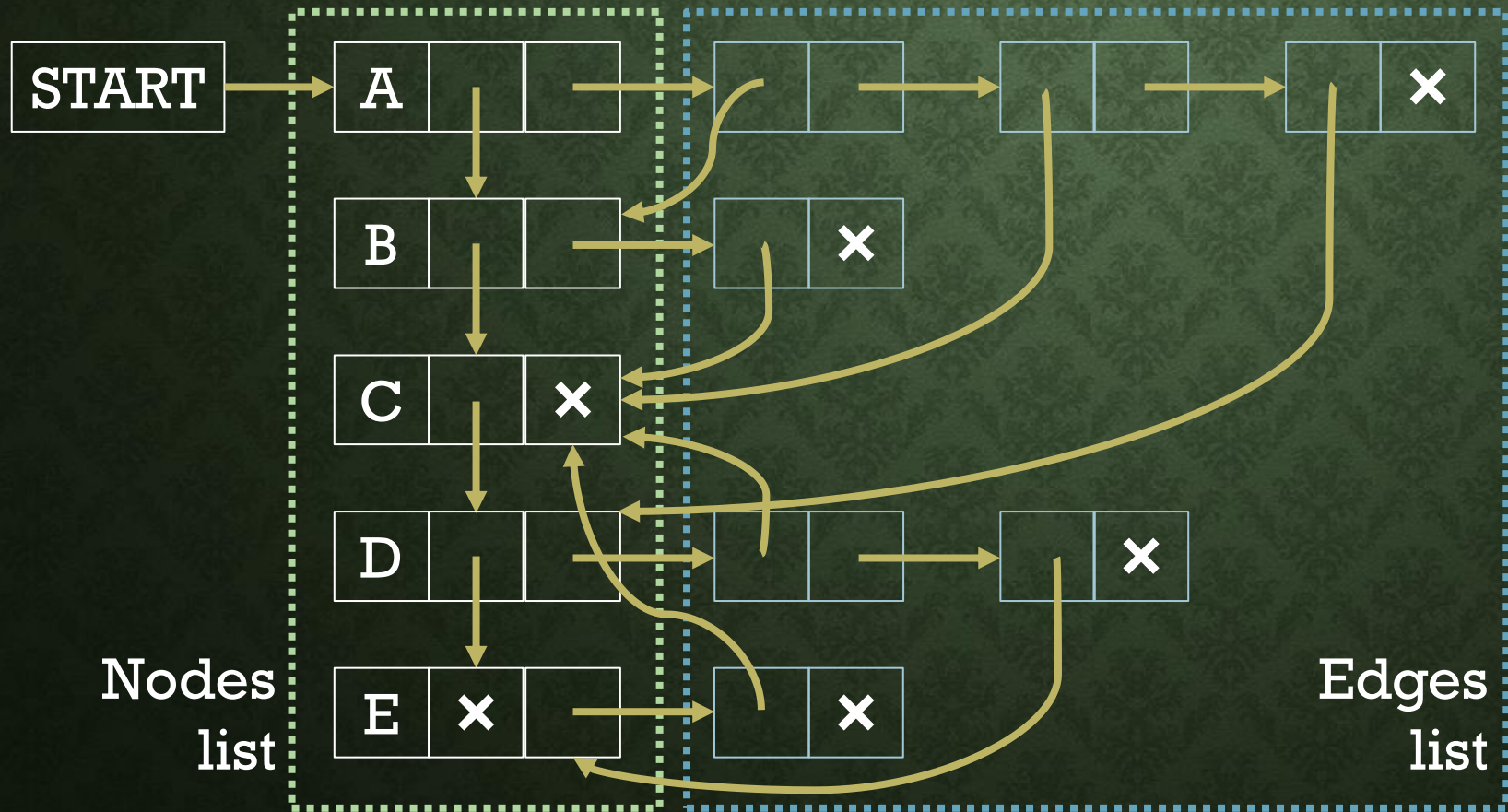


NODE	NEIGHBOURS
A	B, C, D
B	C
C	-
D	C, E
E	C



ADJACENCY LIST

- For each vertex, maintain a list of nodes that are adjacent to it.



NODE	NEIGHBOURS
A	B, C, D
B	C
C	-
D	C, E
E	C



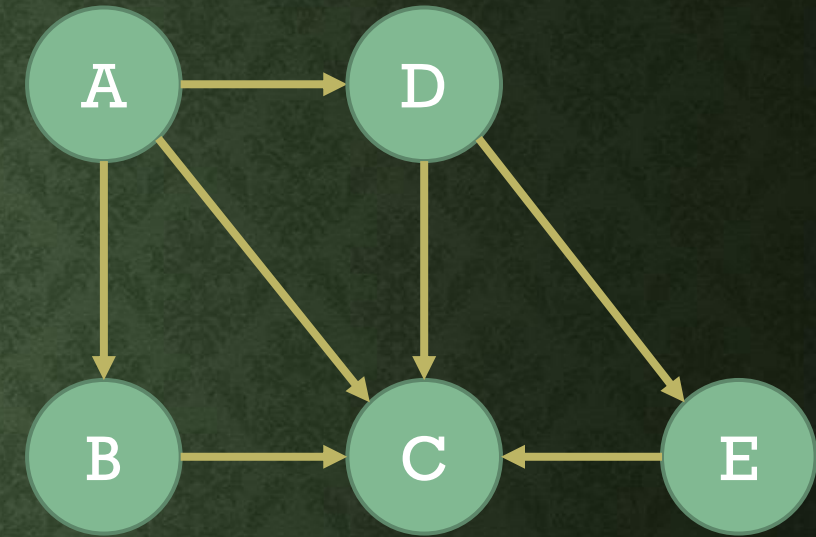
ADJACENCY MATRIX

- Create a Boolean matrix $\mathbb{A}$ of order $n \times n$, where n is the number of nodes in the graph, such that:

$$a_{ij} = \begin{cases} 1 & \text{if } i \text{ is adjacent to } j \\ 0 & \text{otherwise} \end{cases}$$

- The adjacency matrix for the graph shown here is:

$$\mathbb{A} = \begin{matrix} & \begin{matrix} A & B & C & D & E \end{matrix} \\ \begin{matrix} A \\ B \\ C \\ D \\ E \end{matrix} & \begin{bmatrix} 0 & 1 & 1 & 1 & 0 \\ 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 1 \\ 0 & 0 & 1 & 0 & 0 \end{bmatrix} \end{matrix}$$

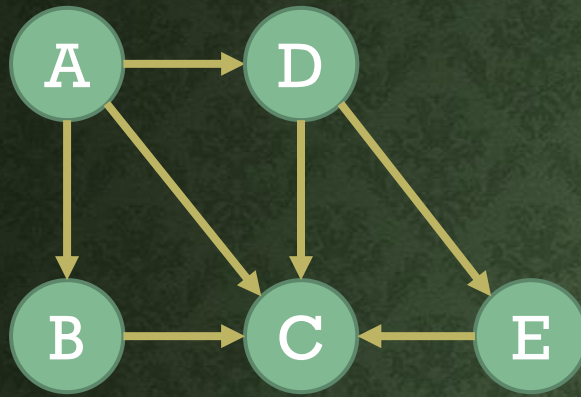


PATH MATRIX

- The adjacency matrix A is the matrix of all paths of length 1.
- A^2 is the matrix of all paths of length 2.
- A^m is the matrix of all paths of length m .
- In a graph of n nodes, the maximum length of a simple path is n . This simple path would be a cycle, if it exists. (Prove this statement.)
- The path matrix P is a matrix of paths of all lengths present in G . It is given as:

$$P = A + A^2 + \dots + A^n$$





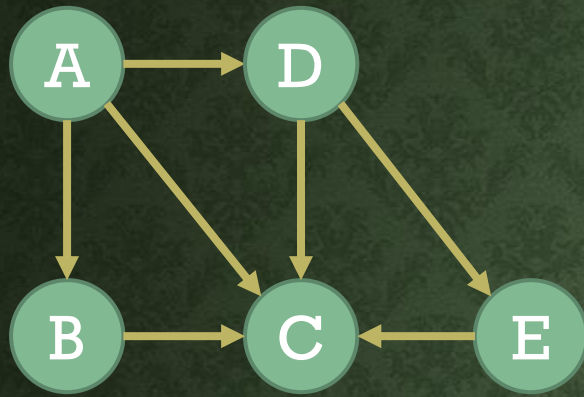
PATH MATRIX

$$A = \begin{bmatrix} 0 & 1 & 1 & 1 & 0 \\ 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 1 \\ 0 & 0 & 1 & 0 & 0 \end{bmatrix}$$

$$A^2 = A \times A = \begin{bmatrix} 0 & 1 & 1 & 1 & 0 \\ 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 1 \\ 0 & 0 & 1 & 0 & 0 \end{bmatrix} \times \begin{bmatrix} 0 & 1 & 1 & 1 & 0 \\ 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 1 \\ 0 & 0 & 1 & 0 & 0 \end{bmatrix} = \begin{matrix} A \\ B \\ C \\ D \\ E \end{matrix} \begin{matrix} A & B & C & D & E \\ \begin{bmatrix} 0 & 0 & 2 & 0 & 1 \\ 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \end{bmatrix} \end{matrix}$$

- There are 2 paths of length 2 from A to C, a path of length 2 from A to E, and a path of length 2 from D to C.





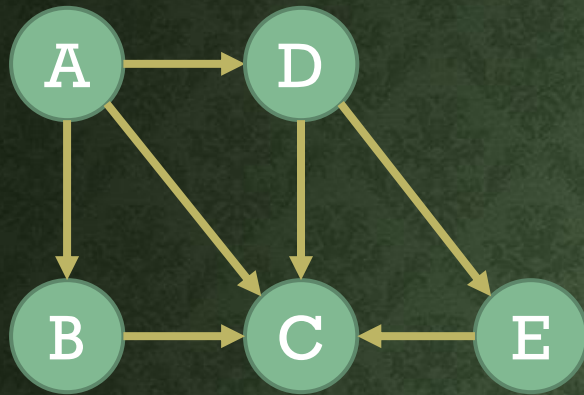
PATH MATRIX

$$A = \begin{bmatrix} 0 & 1 & 1 & 1 & 0 \\ 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 1 \\ 0 & 0 & 1 & 0 & 0 \end{bmatrix}, A^2 = \begin{bmatrix} 0 & 0 & 2 & 0 & 1 \\ 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \end{bmatrix}$$

$$A^3 = A^2 \times A = \begin{bmatrix} 0 & 0 & 2 & 0 & 1 \\ 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \end{bmatrix} \times \begin{bmatrix} 0 & 1 & 1 & 1 & 0 \\ 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 1 \\ 0 & 0 & 1 & 0 & 0 \end{bmatrix} = \begin{matrix} A \\ B \\ C \\ D \\ E \end{matrix} \begin{matrix} A & B & C & D & E \\ \begin{bmatrix} 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \end{bmatrix} \end{matrix}$$

- There is a path of length 3 from A to C.





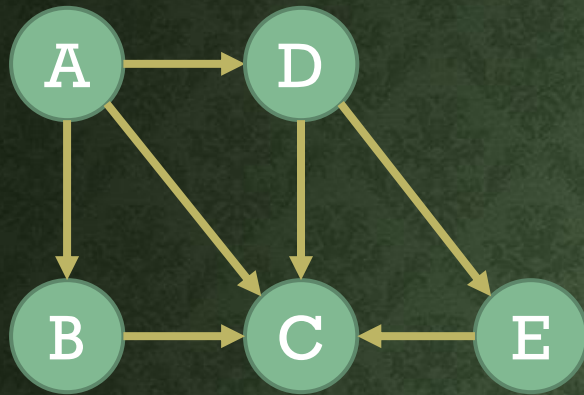
PATH MATRIX

$$A = \begin{bmatrix} 0 & 1 & 1 & 1 & 0 \\ 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 1 \\ 0 & 0 & 1 & 0 & 0 \end{bmatrix}, A^2 = \begin{bmatrix} 0 & 0 & 2 & 0 & 1 \\ 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \end{bmatrix}, A^3 = \begin{bmatrix} 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \end{bmatrix}$$

$$A^4 = A^3 \times A = \begin{bmatrix} 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \end{bmatrix} \times \begin{bmatrix} 0 & 1 & 1 & 1 & 0 \\ 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 1 \\ 0 & 0 & 1 & 0 & 0 \end{bmatrix} = \begin{matrix} A \\ B \\ C \\ D \\ E \end{matrix} \begin{matrix} A & B & C & D & E \\ \begin{bmatrix} 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \end{bmatrix} \end{matrix}$$

- There are no paths of length 4.
- We stop calculating further powers of A when the resultant matrix is all zeroes or the resultant matrix is the same as a lower power of A , previously calculated.





PATH MATRIX

$$\mathbb{A} = \begin{bmatrix} 0 & 1 & 1 & 1 & 0 \\ 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 1 \\ 0 & 0 & 1 & 0 & 0 \end{bmatrix}, \mathbb{A}^2 = \begin{bmatrix} 0 & 0 & 2 & 0 & 1 \\ 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \end{bmatrix}, \mathbb{A}^3 = \begin{bmatrix} 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \end{bmatrix}, \mathbb{A}^4 = 0$$

$$\mathbb{P} = \mathbb{A} + \mathbb{A}^2 + \mathbb{A}^3 = \begin{matrix} & \begin{matrix} A & B & C & D & E \end{matrix} \\ \begin{matrix} A \\ B \\ C \\ D \\ E \end{matrix} & \begin{bmatrix} 0 & 1 & 4 & 1 & 1 \\ 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 2 & 0 & 1 \\ 0 & 0 & 1 & 0 & 0 \end{bmatrix} \end{matrix}$$

- The path matrix shows all paths present in the graph.



HOMEWORK

103	Atlanta to Houston
106	Houston to Atlanta
201	Boston to Chicago
305	Chicago to Miami
308	Miami to Boston
203	Boston to Denver
204	Denver to Boston
301	Denver to Reno
402	Reno to Chicago

Friendly Airlines has 9 daily flights.

Draw a graph showing all flights, representing each location by the first letter of its name.

Store this graph as an adjacency list, storing flight numbers in the edge nodes. (Use three fields in each edge node.)

Compute the adjacency matrix for all flights. Take row and column elements in alphabetical order.

Compute the path matrix for Friendly Airlines.



WHAT DID WE LEARN TODAY?

We were introduced
to graph
terminology.

We stored the
graph in
memory.

We computed
path matrix
for the graph.



THINGS TO DO



Read the book!



Note your questions and put them up in the relevant online session.



Email suggestions on content or quality of this lecture at uroojain@neduet.edu.pk

